MyTime: Making Time Searchable

A Concept Paper for Universal Availability Infrastructure

Aaron Garcia

aaron@garcia.ltd

Version 2.1 | PUBLIC

6 August 2026

Executive Summary

Every resource that depends on time availability — driving instructors, wedding venues, GPU clusters, consultants — faces the same problem: their spare capacity is invisible to the people who need it. Booking platforms address this for specific verticals but extract 10–30% commissions and lock providers into proprietary systems. No open, cross-category availability layer exists.

This paper proposes MyTime, a system for publishing and discovering time availability using a minimal three-state model (Free/Busy/Tentative). The idea is not new in isolation. Calendar interoperability standards — iCalendar (RFC 5545), CalDAV (RFC 4791), and the VFREEBUSY mechanism — already handle availability exchange between known parties. What they do not provide is a public, searchable directory where anyone can discover available resources by time, location, and category without knowing who to ask. MyTime sits in that gap.

This is a concept paper, not a specification. It describes the design principles and open questions that would need resolution before any implementation could begin. We seek contributors willing to challenge the assumptions here and help determine whether the concept survives contact with engineering reality.

1.0 The Problem

A driving instructor finishes a lesson at 2pm. The next booking is at 4pm. Those two hours are available, but no one outside the instructor's existing client base will ever know. A wedding venue has an open Saturday in July — worth thousands of pounds — that sits empty because no couple searching for "venues free in July" can discover it. The information exists. It is trapped in private calendars and vertical booking silos.

This is not a niche complaint. The pattern repeats across every sector where time availability drives transactions: healthcare appointments, conference rooms, freelance consulting, vehicle hire, studio space, computational capacity. In each case, the provider knows when they are free and the seeker knows when they need something. The two cannot find each other efficiently.

Existing booking platforms solve this within verticals. Calendly handles meetings. Treatwell handles salons. Booking.com handles hotels. Each platform builds its own availability index, takes a commission on transactions, and has no incentive to share data with competing platforms. The result is that availability information is balkanised — duplicated across dozens of incompatible systems, each covering a fraction of the market.

The structural failure runs deeper than fragmentation. Current systems are person-centric: you search for a specific provider and then check their availability. The more useful query is time-centric: "who is free at 3pm Tuesday within 10 miles that offers what I need?" That query cannot be answered today: the only cross-provider availability aggregation operating at consumer scale — Google's Reserve programme,

discussed in Section 2.2 — is vertical-limited, partner-gated, and place-centric. Building an open, time-centric index is the purpose of MyTime.

I should be honest about why this has not already been built. A universal availability directory has no obvious revenue model. It does not process transactions, so it cannot take commissions. It does not own the relationship between provider and seeker, so it cannot sell advertising effectively. For venture-backed platforms, this looks like a terrible business. The argument for building it anyway is the same argument that applied to email, DNS, and the early web: some infrastructure works better as a public good. Whether that argument is sufficient to sustain the system economically is an open question — one this paper confronts directly in Section 6.0 without pretending to have resolved.

2.0 Prior Art

MyTime does not emerge from a vacuum. Calendar interoperability is a mature field with decades of standardisation work, and several adjacent systems already operate in parts of the space this paper describes. Any honest proposal must acknowledge what already exists and articulate precisely what it claims to add.

2.1 The Calendar Standards Landscape

iCalendar (RFC 5545) defines the data format for calendar events, including the VFREEBUSY component — a mechanism for publishing blocks of free, busy, or tentative time. This is, at first glance, exactly what MyTime proposes. The critical difference is scope: VFREEBUSY was designed for exchange between known parties (scheduling a meeting between colleagues) rather than public discovery by strangers. It answers “when is this person free?” not “who is free at this time?”

CalDAV (RFC 4791) and **CalDAV Scheduling (RFC 6638)** extend iCalendar with server-based calendar access and scheduling negotiation. These protocols handle calendar synchronisation and meeting requests within organisations or between federated calendar servers. They are powerful but inward-facing — designed for coordination among identified participants, not for broadcasting availability to the open internet.

RFC 7953 (Calendar Availability) defines a VAVAILABILITY component that goes further than VFREEBUSY, allowing resources to express recurring availability patterns (akin to MyTime’s template concept). This is the closest existing standard to what MyTime describes. The gap is that VAVAILABILITY is a data format, not a discovery service. A resource can express “I am generally free 9–5 on weekdays” in VAVAILABILITY, but there is no standardised mechanism for a stranger to search for resources matching that pattern.

2.2 Adjacent Systems

Google Calendar’s Free/Busy API provides cross-user availability queries within a Google Workspace domain. It can answer “when is this list of people free?” but

requires knowing who to ask and does not support public, anonymous, cross-domain discovery. More significant is Reserve with Google (the Google Actions Center): partner booking systems feed real-time availability for restaurants, salons, fitness classes, and appointment services into Google Search and Maps, where users can discover and book open slots. This is the closest live system to what MyTime describes, and it demonstrates that cross-vertical availability aggregation works at consumer scale. It also illustrates the structural limits of a corporate-owned index: participation requires integration through approved booking partners, coverage extends only to commercially attractive verticals, queries remain place-centric ("book a table at X") rather than time-centric ("who is free at 3pm?"), and the index exists to route transactions through Google's ecosystem. Apple's iCloud Calendar is the other major ecosystem. Apple's privacy-first positioning makes them a natural candidate to build availability sharing — if they chose to, iCloud Calendar could offer privacy-preserving availability publication to hundreds of millions of users overnight. That Google's version of this exists only as a partner-gated booking channel, and Apple has not built one at all, is itself informative: a neutral, universal availability index is not something either company has a commercial incentive to create.

Calendly, Cal.com, and similar tools publish individual scheduling pages where visitors can book time. These are closer to MyTime's model — availability is public and strangers can access it — but they operate per-provider, not as a searchable index. You must already know the provider's Calendly link. schema.org/Schedule defines structured data for expressing schedules on web pages, enabling search engines to index availability. Adoption has been slow for scheduling data, but the approach is architecturally relevant: it demonstrates that decentralised, crawl-based availability discovery is technically feasible.

2.3 What MyTime Would Add

The gap in the existing landscape is a searchable, cross-category availability index combining three properties: public discovery (strangers can find resources), time-centric queries (search by availability window, not by provider name), and cross-domain scope (not limited to a single organisation, platform, or vertical). Reserve with Google comes closest, providing public discovery across several verticals, but it is partner-gated, booking-mediated, and controlled by a single company with no obligation to neutrality or universality. The gap MyTime addresses is therefore stated precisely: no open availability index exists — one that any resource can join directly, that intermediates no transactions, and that treats a GPU cluster, a village hall, and a driving instructor identically.

MyTime's proposed contribution is not a new data format — VFREEBUSY and VAVAILABILITY are adequate — but a discovery and indexing layer built on top of existing calendar standards. Whether that layer needs to be a centralised service, a federated network, or a crawl-based index is an open architectural question explored in Section 4.0.

2.4 Conceptual Grounding

The problem MyTime addresses is not purely technical. It sits at the intersection of two well-studied domains.

In marketplace design, the matching of supply and demand under incomplete information is a central problem. Roth's work on market design (Roth, 2002; Roth, 2008) established that market failures often stem from congestion (too many participants trying to transact simultaneously) and from thickness failures (not enough participants to produce good matches). MyTime's cold-start problem is a thickness failure: without enough published availability, searches fail, discouraging adoption. The marketplace design literature offers no silver bullet for bootstrapping, but it does suggest that reducing transaction costs for early participants (rather than subsidising them) is the most sustainable path.

In temporal database theory, the indexing of time-varying data has been studied extensively (Jensen et al., 1998; Salzberg and Tsotras, 1999). Temporal range queries — "find all resources where state=Free overlaps the interval [14:00, 16:00] on 2026-03-15" — are well-understood in the database literature, with R-tree and interval-tree structures providing efficient solutions. The technical feasibility of temporal search at scale is not in question; the indexing primitives exist. What is missing is the data, not the query capability.

3.0 The Design

3.1 Three States

The atomic unit of availability in MyTime is deliberately minimal:

State	Value	Meaning
Free	0	Available for booking or allocation
Busy	1	Unavailable, confirmed
Tentative	2	Provisionally booked or lower priority

Three states and nothing else. No appointment details, no customer data, no transaction history. This constraint is the foundation of the entire design — it makes privacy preservation architectural rather than policy-dependent, and it makes the system simple enough to apply to any resource type from yoga studios to container ships.

The storage model exploits a useful asymmetry: most resources are in their default state most of the time. A driving instructor who works 40 hours a week is Free for the remaining 128 hours. If Free is the default, only those 40 Busy hours need recording. For resources with heavily skewed availability — and most real-world resources have exactly this property — the storage savings are significant. For resources that are roughly equally split between states, the savings are minimal. The design claims efficiency in the common case, not universally.

There is also a question this simplicity raises but does not resolve: who arbitrates when two calendar sources disagree about a resource's state? If a corporate calendar

says Busy and a personal calendar says Free for the same time slot, what does MyTime publish? The simplest rule — last write wins from the resource owner — handles most cases but may not survive complex multi-source aggregation. Section 3.3 proposes a strawman for conflict resolution.

3.2 Variable Granularity

Time means different things to different resources. A music tutor works in 30-minute blocks. A wedding venue books in half-days. A cloud compute provider allocates by the minute. Any system that forces a single time quantum will serve some resources well and others badly.

MyTime allows each resource to define its own granularity, from 10 minutes to 180 days. Searches that span different granularities return results sorted by compatibility with the requested duration. A search for “2 hours” would rank a resource with 30-minute granularity (good fit) above one with half-day granularity (wasteful match), while a search for “full Saturday” would reverse that ranking. The exact scoring mechanism is an implementation detail that depends on the query engine design.

3.3 Templates, Recurrence, and Conflict Resolution

Rather than storing individual time cells for months ahead, resources define templates: recurring patterns that generate availability on demand. This concept maps directly to RFC 7953’s VAVAILABILITY component, and any implementation should build on that standard rather than reinventing it.

A driving instructor’s template might express: “weekdays, 08:00–18:00, state=Free, granularity=30min” with exceptions for lunch and a recurring Wednesday afternoon off. This handful of rules projects availability indefinitely. Explicit overrides — a bank holiday, a cancelled lesson, a one-off booking — layer on top.

Previous drafts of this paper flagged template conflict resolution as an open question. Here is a strawman proposal for how it could work. Templates are versioned and immutable once published; modifications create a new version rather than editing in place. When the system generates time cells for a query, it evaluates rules in precedence order: explicit per-slot overrides first, then the most recently published template version, then older template versions. For overlapping rules at the same precedence level, the more specific rule wins (a rule for “Wednesday 14:00–15:00” beats a rule for “weekdays 09:00–17:00”). Template modifications apply only to future time — they do not retroactively alter availability that has already been queried or acted upon. If a resource changes timezone, all future template projections shift to the new timezone from the point of change; historical data retains its original UTC timestamps.

This is a strawman, not a specification. The semantics are plausible but untested against edge cases that a real implementation would surface.

3.4 Calendar Aggregation and Non-Digital Providers

People split their lives across multiple calendar systems. Work schedules live in Exchange or Google Workspace. Personal time lives in Apple Calendar or a paper diary. Side businesses might use Calendly or a spreadsheet. MyTime's aggregation layer would ingest multiple sources into a unified availability profile.

This design assumes that providers have digital calendars. Many do not. A sole-trader plumber, a village hall hire manager, a private music teacher — these are exactly the kinds of resources MyTime aims to serve, and many of them manage availability on paper, in their heads, or via WhatsApp messages. For these providers, MyTime must offer a direct input mechanism: a simple web or mobile interface where they can set their availability manually, without needing an existing calendar system to feed from. The aggregation layer is a convenience for digitally-equipped providers, not a prerequisite for participation.

On profile separation: the earlier v2 draft claimed that separate profiles for the same person would not "reveal the existence of the other." That claim was too strong. Profile separation provides presentation-layer isolation, not cryptographic unlinkability. If a single account controls both profiles, the correlation is trivial for anyone with access to the account registry. Users who need genuine separation would need to register profiles independently, using separate credentials.

3.5 A Strawman Integration Architecture

Previous reviews noted that the paper describes features without showing how data flows through the system. Here is a simplified walkthrough of how a resource's availability would move from source to search result.

Step 1: Publication. A resource owner publishes availability to MyTime through one of three channels: (a) a CalDAV or iCalendar feed that MyTime polls periodically, (b) a REST API that the owner's system pushes updates to, or (c) direct entry through MyTime's own web/mobile interface. All three channels produce the same internal representation: a resource identifier, a set of template rules, and a set of explicit overrides, all expressed in terms of the three-state model.

Step 2: Indexing. MyTime's indexing engine processes the published data into a query-optimised temporal index. For each resource, the engine materialises template projections into concrete time cells for a rolling window (say, 90 days forward) and merges them with explicit overrides using the precedence rules from Section 3.3. The materialised cells are stored in state-segregated indices: one for Free, one for Busy, one for Tentative. Resources in their default state are absent from all indices (zero-storage default).

Step 3: Query. A seeker submits a query specifying a time window, a duration, optional location constraints, and optional category filters. The query engine searches the Free index for resources with matching availability, applies granularity scoring, and returns ranked results. Each result includes the resource's public profile (name, category, location, contact/booking link) and the matching availability windows. No audit trail data, no state-change history, no information about other time slots.

Step 4: Engagement. The seeker follows the resource's contact link to engage directly. MyTime has no further involvement. It does not confirm bookings, process payments, or track outcomes.

This is a conceptual data flow, not an API specification. Wire formats, authentication mechanisms, error handling, and consistency guarantees would all need definition in a technical specification.

3.6 Storage and Query Architecture

For a system designed to serve millions of resources, query performance matters. The state-segregated storage model (separate indices for Free, Busy, and Tentative) means that availability searches touch only the relevant index. The performance characteristics depend entirely on the underlying database technology and index design — temporal range queries are well-served by R-tree or interval-tree structures (Salzberg and Tsotras, 1999), but specific latency targets would be premature without benchmarks against a working prototype.

All times are stored in UTC with timezone metadata per resource. Mobile resources — delivery vehicles, ships, aircraft — update location data affecting timezone interpretation. An audit trail records state changes with timestamps and the identity of the actor making the change. Audit trails are visible only to the resource owner. Aggregate availability data is visible to searchers. Individual state-change histories are not exposed through the query interface.

4.0 Open Design Questions

A production system would require answers to questions that remain open at the concept stage. Some of these are straightforward engineering; others are genuinely hard problems where the right answer depends on real-world adoption patterns that cannot be predicted in advance.

4.1 Trust: Identity, Ownership, and Spam

Three related problems cluster under "trust," and they are worth discussing together because they share a common tension: openness versus integrity.

Who can update a resource's state? The simplest model is API key authentication — whoever holds the key controls the resource. For providers with existing websites, domain verification (proving you control a URL associated with the resource) adds a layer of credibility. Business registry lookup is more robust but introduces friction that would deter small providers. The right balance depends on the threat environment, which in turn depends on whether MyTime attracts enough usage to be worth attacking.

Spam is the harder problem. An open publication system is vulnerable to fake resources publishing fictitious availability to pollute search results, game rankings, or suppress competitors. Reputation scoring is the natural response, but MyTime faces a specific handicap: if engagement happens outside the system, there is no direct signal

of whether published availability leads to real bookings. Verification-based ranking — where resources that prove identity (domain ownership, business registry, trade association membership) rank above unverified ones — is a partial mitigation. It does not eliminate fake listings but makes them less visible. Whether this is sufficient depends on attacker motivation and scale, which can only be assessed after launch.

4.2 Federation, Centralisation, and the Architecture Decision

This is the hardest open question in the paper, and it deserves more space than the others because the choice shapes everything downstream: governance, consistency, cost, and the contributor profile needed to build it.

A centralised service is the fastest to build. One database, one query engine, one API. Consistency is trivial — there is only one source of truth. The cost is a single point of failure, a governance problem (who controls the index?), and a scaling challenge as the system grows beyond what one team can operate. For a concept at the proof-of-concept stage, centralisation is pragmatic. If MyTime reaches the scale where centralisation becomes a liability, it will have also reached the scale where federation becomes fundable.

Federation (multiple cooperating servers, like email) distributes control and eliminates the single point of failure, but introduces consistency challenges. A resource might be Free on one server and Busy on another during propagation delays. Calendar systems already handle this through CalDAV's synchronisation mechanisms, so the problem is not unsolved, but it adds engineering complexity and requires a coordination protocol between servers.

A crawl-based model (like web search) avoids coordination entirely. Resources publish structured availability data on their own infrastructure — perhaps using schema.org/Schedule markup or a MyTime-specific format — and MyTime crawls and indexes it. This is architecturally elegant and fully decentralised, but it depends on every provider maintaining their own web presence with structured data, which is a high bar for small providers and impossible for those without a website.

The pragmatic path is likely: start centralised, design the data model to be federation-compatible, and migrate to a federated or crawl-based model if adoption warrants it. But this must be an explicit design decision, not an accident.

4.3 Governance

Any infrastructure layer that aspires to neutrality needs a governance model that prevents capture by a single commercial entity. If MyTime is built as open-source software with an open specification, governance could follow models like the Apache Foundation or the CalConnect consortium. If it operates as a hosted service, it needs a structure that guarantees continued openness — a non-profit foundation or a cooperative model.

The absence of a governance commitment is a weakness. Addressing it is a prerequisite for attracting serious technical contributors.

5.0 The Cold-Start Problem

Every two-sided system faces this: providers will not publish availability until seekers are searching, and seekers will not search until providers have published. There is no way to pretend this is easily solved.

5.1 Why It Is Hard

Unlike email (which is useful with two participants) or DNS (which migrated existing domain registrations from day one), MyTime has no intermediate value at low adoption. A search that returns zero results is worse than useless — it teaches the searcher not to come back. The system needs a critical mass of resources in a specific category and geography before any individual search produces useful results.

5.2 The Widget Strategy

The most promising bootstrapping path is one that offers value to providers before the search network exists.

Consider a free, embeddable availability widget: a small component that any resource provider can add to their existing website showing “see my live availability.” The widget pulls data from the provider’s MyTime profile — which they populate either by linking a calendar feed or by entering availability directly. The widget works immediately and independently: it is useful on day one, with zero seekers using the MyTime search index, because it serves the provider’s existing visitors.

The strategic value is that every widget deployed feeds the central MyTime index as a side effect. The provider gets a free scheduling tool for their website. MyTime gets data. As widget adoption grows across a vertical and geography, the search index quietly accumulates density. At some threshold, the search product becomes viable — not as a launch, but as a natural consequence of widget adoption.

This sidesteps the classic cold-start trap. The provider’s initial value proposition is the widget, not the network. The network is an emergent property, not a prerequisite.

The widget strategy also addresses non-digital providers. A plumber without a website can use the MyTime mobile app as their primary availability tool — setting slots, sharing a booking link with customers — without needing to understand or care about the search index running behind it.

5.3 Other Strategies

Vertical seeding. Partner with a trade association in a single industry — say, driving instruction in the UK, where roughly 40,000 approved instructors operate. If the association promotes MyTime widget adoption to its members, density in that vertical could reach critical mass within a single geography. The challenge: this requires partnership development, which takes time and is not a technical problem.

Calendar integration. If MyTime can read existing calendar data via CalDAV or API feeds, providers do not need to actively publish — their existing calendar becomes the data source. This is technically feasible but raises a serious consent question. Availability derived from a private calendar must not be published without explicit, informed, opt-in consent from the resource owner. Under GDPR (and UK GDPR), calendar data may constitute personal data — particularly if availability patterns reveal information about the data subject's behaviour (e.g., "always busy on Thursdays" implies a recurring commitment). Any calendar ingestion feature must present a clear consent flow explaining exactly what will be published, provide granular controls (the resource owner can exclude specific calendars or time ranges), and ensure that only three-state availability — not event titles, attendee lists, or descriptions — is ever extracted from the source calendar. A "we read your calendar and publish your availability" feature without robust consent design would be both a legal liability and a trust-destroying mistake.

5.4 Degradation

MyTime degrades catastrophically on the search side: below critical mass, every empty result erodes trust. The widget strategy provides graceful degradation on the provider side, because the widget is useful regardless of search volume. This asymmetry is the foundation of the bootstrapping plan: grow provider-side adoption through widget value, and let search-side value emerge once density is sufficient. Premature launch of the search product with sparse data would be worse than no launch at all.

6.0 Economic Sustainability

A zero-commission discovery system cannot fund itself through transaction fees. This is a feature (no platform rent-seeking) and a vulnerability (no obvious revenue).

What must be free: Publishing availability and searching for available resources. These are the core functions, and charging for either would undermine adoption.

What could carry fees: Premium features for resource owners (analytics on search impressions, verified badges, priority placement in results). Commercial API access for high-volume integrators (booking platforms querying MyTime to show availability within their own interfaces). Historical audit data access beyond a basic retention window.

What remains unproven: Whether any of these fee-bearing services would generate sufficient revenue to cover infrastructure costs at scale. The honest position is that economic sustainability is an unsolved constraint. Institutional sponsorship — from workforce development agencies, healthcare systems, or transport authorities that benefit from reduced scheduling friction — is a candidate model worth exploring, but no institution has been approached. The widget strategy offers a secondary revenue path: if the widget becomes valuable enough that providers want premium features (custom branding, booking integration, analytics), those features could be monetised while keeping the core widget and search index free.

The risk is real: if the system cannot fund itself, it dies regardless of how good the concept is. Any implementation plan must include a financial sustainability workstream from day one, not as an afterthought.

7.0 Use Cases

These scenarios illustrate how MyTime would function at sufficient adoption. They are intended to make the concept concrete, not to demonstrate that these outcomes are certain — each depends on solving the cold-start and sustainability problems described above.

A driving instructor in Manchester adds the MyTime widget to her website. Students visiting her site see live availability and can identify open slots without phone calls. Independently, a student new to the area searches “driving lesson, weekday afternoon, M20 postcode” on the MyTime search page and sees slots from eight instructors, sorted by next available. Both interactions use the same underlying data; the instructor set it up for the widget and got search visibility as a bonus.

A couple searching for a summer wedding venue queries “4-hour block, Saturday, June through September, within 50 miles of Bristol.” Results include a castle, two hotels, a garden centre, and a village hall — resources that would never appear together on any single booking platform because they operate in different verticals. The couple visits each venue’s website to evaluate pricing and facilities.

A cloud computing provider publishes spot capacity for GPU clusters. A research lab searches “8 continuous hours of GPU time, starting after 6pm, under 2 USD per GPU-hour.” The search is temporal and capability-filtered, finding underutilised capacity across multiple providers. The lab contacts each provider directly through the listed API endpoint.

These are plausible, not proven. Each depends on provider density that does not yet exist.

8.0 Discussion and Limitations

The core idea — time-centric discovery across resource categories — has merit. The three-state model is genuinely minimal and privacy-preserving. The argument that availability should be a searchable public resource rather than a proprietary asset is philosophically sound. The widget bootstrapping strategy offers a plausible (though unproven) path through the cold-start problem.

The weaknesses are significant. No wire protocol, message format, or API schema has been defined. No consistency model has been chosen. No governance structure exists. The economic model is aspirational. The marketplace design literature (Roth, 2002) warns that thickness failures — insufficient participants to produce good matches — are the most common cause of market failure, and MyTime is squarely exposed to this risk.

The competitive landscape deserves candid assessment. Google already operates Reserve with Google — a partner-gated availability and booking channel across several verticals (Section 2.2) — alongside free/busy lookups within Workspace domains. Extending Reserve into an open, time-centric index would be a smaller step for Google than building MyTime from scratch would be for anyone else. Apple's iCloud Calendar has hundreds of millions of users and a privacy-first brand that would make availability sharing a natural feature extension. If either company chose to build a truly public availability directory, they would have the user base, the calendar integration, and the infrastructure to execute overnight. MyTime's argument for existing independently is that availability infrastructure should not be controlled by a single corporation — the same argument made for open email versus proprietary messaging, for open web versus walled-garden apps. That argument is principled. It is also, historically, a fight that open systems sometimes lose.

The strongest version of the case for MyTime is not that it will definitely succeed, but that the concept deserves to be tested. If the widget strategy generates provider adoption, if a vertical-geography deployment reaches critical mass, and if the economic model proves even marginally sustainable, then the case for a full technical specification becomes strong. If any of those conditions fails, the concept was worth exploring and the cost of finding out was low.

9.0 Next Steps

This paper is a starting point for a conversation, not a blueprint for a build. The immediate priorities are finding people who will stress-test the concept and building a working prototype of the availability widget to validate the bootstrapping hypothesis.

The contributors needed are not cheerleaders. They are calendar protocol engineers who can say whether VAVAILABILITY and CalDAV Scheduling already solve this problem (making MyTime redundant), distributed systems architects who can evaluate whether the storage and query model is viable at scale, economists who can assess whether a zero-commission infrastructure layer can sustain itself, and privacy engineers who can design GDPR-compliant calendar ingestion.

If the concept survives that scrutiny, the next deliverable is a technical specification: wire protocols, API schemas, consistency models, and a governance charter. That specification would be a different document entirely.

The web made information searchable. MyTime makes time searchable.

Contact: aaron@garcia.ltd

Appendix A: Document Metadata

Document Title	MyTime: Making Time Searchable

Subtitle	A Concept Paper for Universal Availability Infrastructure
GUID	3feff7bc-68fb-4bed-ba97-d9bcae3800e5
Version	2.1
Date	6 August 2026
Author	Aaron Garcia
Affiliation	Independent Researcher
Contact	aaron@garcia.ltd
Classification	PUBLIC
Status	RELEASED

Appendix B: Glossary

Term	Definition
API	Application Programming Interface
CalDAV	Calendar Distributed Authoring and Versioning — an Internet standard for calendar access (RFC 4791)
DNS	Domain Name System — the Internet's naming service
EWS	Exchange Web Services — Microsoft's API for Exchange Server calendar and mail access
GDPR	General Data Protection Regulation — EU/UK data protection law
GPU	Graphics Processing Unit — high-performance computing hardware
HTTP	Hypertext Transfer Protocol — the foundation of web data communication
iCal / iCalendar	A standard file format (.ics) for calendar data exchange (RFC 5545)
REST	Representational State Transfer — an architectural style for networked APIs
RFC	Request for Comments — a publication describing Internet methods, standards, and protocols
Sybil Attack	A security threat where a single entity creates multiple fake identities to gain disproportionate influence
UTC	Coordinated Universal Time — the primary global time standard
VAVAILABILITY	An iCalendar component for expressing recurring availability patterns (RFC 7953)
VFREEBUSY	An iCalendar component for expressing blocks of free/busy time (RFC 5545)

Appendix C: Methodology and Transparency Statement

Author Context and Expertise

This work is grounded in 30 years of multi-sector experience in Systems Engineering. The analytical framework reflects professional history, focusing on practical application.

Digital Methodology and Accessibility

In the spirit of transparency, I utilised a suite of Generative AI tools — specifically Claude, Google Gemini, and ChatGPT — to assist in the production of this paper.

These tools were employed as “force multipliers” for data synthesis and editorial accessibility. As a writer with dyslexia, I leverage these models as assistive technology to refine grammar and streamline sentence structure.

Verification and Integrity

While AI assisted in processing data, the intellectual oversight is entirely human. I performed manual validation of all citations and accept full responsibility for the accuracy and originality of the final output.

Appendix D: References

Standards

Ref	Standard
RFC 4791	Calendaring Extensions to WebDAV (CalDAV)
RFC 5545	Internet Calendaring and Scheduling Core Object Specification (iCalendar)
RFC 6638	Scheduling Extensions to CalDAV
RFC 7953	Calendar Availability (VAVAILABILITY)

Academic Literature

Citation	Reference
Jensen et al., 1998	C.S. Jensen, R.T. Snodgrass, et al. “The Consensus Glossary of Temporal Database Concepts.” Temporal Databases: Research and Practice, Springer, 1998.
Roth, 2002	A.E. Roth. “The Economist as Engineer: Game Theory, Experimentation, and Computation as Tools for Design Economics.” Econometrica, 70(4), 2002.
Roth, 2008	A.E. Roth. “What Have We Learned from Market Design?” The Economic Journal, 118(527), 2008.
Salzberg and Tsotras, 1999	B. Salzberg and V.J. Tsotras. “Comparison of Access Methods for Time-Evolving Data.” ACM Computing Surveys, 31(2), 1999.